1. (3 Punkte) Ein Keller k ist zu Beginn leer. Notiere nach jeder Operation den Inhalt des Kellers horizontal mit dem top-Element auf der linken Seite. Welche Ausgabe erscheint auf der Konsole?

```
k.push(4)
k.push(9)
print(k.top())
k.pop()
k.push(2)
k.push(7)
k.pop()
print(k.top())
```

**Lösung:**

```
k.push(4)          4
k.push(9)          9 4
print(k.top())     9 4       Ausgabe 9
k.pop()            4
k.push(2)          2 4
k.push(7)          7 2 4
k.pop()            2 4
print(k.top())     2 4       Ausgabe 2
```

2. (2 Punkte) a. Welche Datenstruktur wird hier verwendet (Liste/Keller/Schlange)?
   b. Was erscheint auf der Konsole?

```
a = []
for i in range(5):
    a.append(i)

print(a.pop(), end=' ')
print(a.pop(), end=' ')

for i in range(6,9):
    a.append(i)

while a:
    print(a.pop(), end=' ')
```

**Lösung:**

a. Keller b. `4 3 8 7 6 2 1 0`

3. (2 Punkte) a. Welche Datenstruktur wird hier verwendet (Liste/Keller/Schlange)?
   b. Was erscheint auf der Konsole?

```
a = []

for k in range(3):
    for i in range(3):
        a.append(i)
    print(a.pop(), end = ' ')

while a:
    print(a.pop(), end = ' ')
```

**Lösung:** `a. Keller b. 2 2 2 1 0 1 0 1 0`

4. (4 Punkte) Die Dualdarstellung einer Zahl $n$ erhält man, indem man $n$ wiederholt durch 2 teilt und sich die Reste merkt. Der zuerst berechnete Rest ist die letzte Ziffer. Die Funktion `dualzahl(n)` bekommt eine ganze Zahl $n \geq 0$ und gibt die Dualdarstellung als String zurück. Benutze für die Reste eine Python-Liste als Keller. Die Funktion `bin` darf nicht benutzt werden.

```
print(dualzahl(6), dualzahl(37), dualzahl(0))

110 100101 0
```

**Lösung:**

```
def dualzahl(n):
    if n == 0:
        return '0'
    k = []
    while n > 0:
        k.append(n % 2)
        n = n // 2
    s = ''
    while k:
        s += str(k.pop())
    return s
```

5. (5 Punkte) Ein Browser merkt sich die besuchten Seiten in einem Keller. Implementiere die Klasse Browser, das Attribut `verlauf` ist eine Python-Liste.

   - Dem Konstruktor wird die Startseite übergeben.
   - `aktuell()` gibt die aktuelle Seite zurück.
   - `besuche(seite)` öffnet eine neue Seite.
   - `zurueck()` geht zur vorherigen Seite zurück. Auf der Startseite bewirkt die Methode nichts.

   Folgender Dialog soll möglich sein:

```
b = Browser('start.de')
b.besuche('a.de')
b.besuche('b.de')
b.zurueck()
print(b.aktuell())
b.zurueck()
b.zurueck()
print(b.aktuell())

a.de
start.de
```

**Lösung:**

```
class Browser:
    def __init__(self, start):
        self.verlauf = [start]

    def aktuell(self):
        return self.verlauf[-1]

    def besuche(self, seite):
        self.verlauf.append(seite)

    def zurueck(self):
        if len(self.verlauf) > 1:
            self.verlauf.pop()
```

6. (3 Punkte) Gegeben ist das folgende Programm.

```
def a(x):
    return b(x) + 1

def b(x):
    return c(x) * 2

def c(x):
    return x - 3

print(a(5))
```

   (a) (1 Punkt) Notiere den Aufrufstapel, während die Funktion c läuft, horizontal mit dem top-Element auf der linken Seite.

   (b) (1 Punkt) Welche Funktion wird als erste vom Aufrufstapel entfernt?

   (c) (1 Punkt) Welche Ausgabe erscheint auf der Konsole?

**Lösung:** a) `c b a`
b) c, denn sie wurde als letzte aufgerufen (LIFO).
c) 5

7. (3 Punkte) Eine Schlange s ist zu Beginn leer. Notiere nach jeder Operation den Inhalt der Schlange horizontal mit dem Kopf der Schlange auf der linken Seite. Welche Ausgabe erscheint auf der Konsole?

```
s.enq(4)
s.enq(9)
print(s.front())
s.deq()
s.enq(2)
s.enq(7)
s.deq()
print(s.front())
```

**Lösung:**

```
s.enq(4)          4
s.enq(9)          4 9
print(s.front())  4 9      Ausgabe 4
s.deq()           9
s.enq(2)          9 2
s.enq(7)          9 2 7
s.deq()           2 7
print(s.front())  2 7      Ausgabe 2
```

8. (2 Punkte) a. Welche Datenstruktur wird hier verwendet (Liste/Keller/Schlange)?
   b. Was erscheint auf der Konsole?

```
from collections import deque

a = deque([])
for i in range(5):
    a.append(i)

print(a.popleft(), end = ' ')
print(a.popleft(), end = ' ')

for i in range(6,9):
    a.append(i)

while a:
    print(a.popleft(), end = ' ')
```

**Lösung:**
a. Schlange b. `0 1 2 3 4 6 7 8`

9. (2 Punkte) a. Welche Datenstruktur wird hier verwendet (Liste/Keller/Schlange)?
   b. Was erscheint auf der Konsole?

```
from collections import deque
a = deque([])

for k in range(3):
    for i in range(3):
        a.append(i)
    print(a.popleft(), end = ' ')

while a:
    print(a.popleft(), end = ' ')
```

**Lösung:** `a. Schlange b. 0 1 2 0 1 2 0 1 2`

10. (3 Punkte) a. Welche Datenstruktur wird hier verwendet (Liste/Keller/Schlange)?
    b. Was erscheint auf der Konsole?

```
from collections import deque
a = deque([])

for i in range(3):
    for j in range(2):
        a.append(j)
    a.append(i)

while a:
    print(a.popleft(), end = ' ')
```

**Lösung:** `a. Schlange b. 0 1 0 0 1 1 0 1 2`

11. (4 Punkte) Kinder stehen im Kreis und zählen reihum ab. Das k-te Kind scheidet aus, das nächste Kind beginnt wieder bei 1. Die Funktion `abzaehlen(namen, k)` bekommt eine Liste von Namen und gibt die Liste der Namen in der Reihenfolge des Ausscheidens zurück. Benutze eine Schlange (`deque`): Ein Kind, das nicht ausscheidet, stellt sich hinten wieder an.

```
print(abzaehlen(['Ali', 'Bea', 'Cem', 'Dan'], 2))
```

```
['Bea', 'Dan', 'Cem', 'Ali']
```

**Lösung:**

```
from collections import deque

def abzaehlen(namen, k):
    q = deque(namen)
    raus = []
    while q:
        for i in range(k - 1):
            q.append(q.popleft())
        raus.append(q.popleft())
    return raus
```

12. (5 Punkte) Die Klasse Mensa verwaltet die Warteschlange an der Essensausgabe im Attribut `schlange` (eine `deque`).

    - `kommt(name)` stellt eine Person hinten an.
    - `bedient()` entfernt die vorderste Person und gibt ihren Namen zurück. Ist die Schlange leer, wird None zurückgegeben.
    - `vor(name)` gibt zurück, wie viele Personen vor name stehen. Steht name nicht in der Schlange, wird -1 zurückgegeben.

    Implementiere die Klasse Mensa. Folgender Dialog soll möglich sein:

```
m = Mensa()
m.kommt('Ali')
m.kommt('Bea')
m.kommt('Cem')
print(m.vor('Cem'), m.vor('Dan'))
print(m.bedient(), m.vor('Cem'))
```

```
2 -1
Ali 1
```

**Lösung:**

```
from collections import deque

class Mensa:
    def __init__(self):
        self.schlange = deque()

    def kommt(self, name):
        self.schlange.append(name)

    def bedient(self):
        if not self.schlange:
            return None
        return self.schlange.popleft()

    def vor(self, name):
        for i in range(len(self.schlange)):
            if self.schlange[i] == name:
                return i
        return -1
```

13. (3 Punkte) Die Funktion `umdrehen(q)` bekommt eine Schlange q (eine `deque`) und kehrt die Reihenfolge ihrer Elemente um. Als Hilfe darf nur ein Keller (eine Python-Liste) benutzt werden. Implementiere die Funktion.

**Lösung:**

```
def umdrehen(q):
    k = []
    while q:
        k.append(q.popleft())
    while k:
        q.append(k.pop())
```

14. (2 Punkte) Liste, Keller und Schlange sind Klassen mit den bekannten Operationen als Methoden. Welche Zahlenfolge findet sich in den Listenelementen?

```
a = Liste()
k = Keller()
s = Schlange()

for i in range(5):
    s.enq(i)
    k.push(5+i)
for i in range(5):
    a.insert(k.top())
    a.insert(s.front())
    k.pop()
    s.deq()
```

**Lösung:** `4 5 3 6 2 7 1 8 0 9`

15. (3 Punkte) Keller oder Schlange? Entscheide und begründe kurz.
    (a) (1 Punkt) Anrufe bei einer Hotline werden in der Reihenfolge des Eingangs angenommen.
    (b) (1 Punkt) In einem Textprogramm macht „Rückgängig“ die letzte Änderung ungültig.
    (c) (1 Punkt) Beim Ablauf eines Programms merkt sich der Rechner die noch nicht beendeten Funktionsaufrufe.

**Lösung:** a) Schlange. Wer zuerst angerufen hat, ist zuerst dran (FIFO).
b) Keller. Die zuletzt gemachte Änderung wird zuerst zurückgenommen (LIFO).
c) Keller. Die zuletzt aufgerufene Funktion wird zuerst beendet (LIFO).

16. (2 Punkte) Eine Python-Liste eignet sich gut als Keller, aber schlecht als Schlange. Begründe.

**Lösung:** Beim Keller wird nur am Ende der Liste eingefügt und entfernt, dafür muss nichts verschoben werden, das braucht $O(1)$. Eine Schlange muss vorn entfernen. `pop(0)` verschiebt alle übrigen Elemente und braucht $O(n)$.

17. (4 Punkte) Erkläre in ein bis zwei Sätzen.
    (a) (1 Punkt) Was bedeutet LIFO?
    (b) (1 Punkt) Was bedeutet FIFO?
    (c) (1 Punkt) Warum nennt man Keller und Schlange Abstrakte Datentypen?
    (d) (1 Punkt) Was ist der Unterschied zwischen der Operation pop des ADT Keller und der Methode pop einer Python-Liste?

**Lösung:** a) Last in, First out: Das zuletzt hinzugefügte Element wird als erstes entfernt.
b) First in, First out: Das zuerst hinzugefügte Element wird als erstes entfernt.
c) Festgelegt sind nur die Operationen und was sie leisten, nicht die Implementierung.
d) Die Operation pop entfernt das oberste Element nur. Die Python-Methode gibt es außerdem zurück.